

Guia de Configuracion de OpenCode

Preparacion de un proyecto con agentes, skills e instrucciones locales

Documento generado a partir de la guia Markdown del repositorio

Proyecto 2526_PRO_U8_mongo

8 de mayo de 2026

Índice

Guía Paso a Paso para Configurar OpenCode en un Proyecto	3
Objetivo	3
Qué vas a aprender	3
Requisitos	3
Idea clave	4
Estructura mínima recomendada	4
Qué hace cada elemento	4
AGENTS.md	4
.opencode/opencode.json	5
.opencode/agents/	5
.opencode/skills/	5
.opencode/instructions/	5
Paso 1. Crear AGENTS.md	6
Buenas prácticas para AGENTS.md	6
Paso 2. Crear .opencode/opencode.json	6
Paso 3. Entender los parámetros de opencode.json	8
\$schema	8
model	8
default_agent	8
permission	9
watcher.ignore	9
skills.paths	9
instructions	9
agent	9
Paso 4. Crear un agente	10
Qué partes debe tener un agente	10
mode en agentes	11
Paso 5. Crear una skill	11
Qué debe tener una skill	11
Diferencia entre agente y skill	12
Paso 6. Añadir instrucciones complementarias	12
Paso 7. Arrancar OpenCode en el proyecto	12
Abrir la interfaz en el proyecto actual	12
Ejecutar una petición directa desde terminal	12
Ejecutar con un agente concreto	12
Paso 8. Comprobar que OpenCode ha cargado la configuración	13
Ver los agentes disponibles	13
Ver la configuración resuelta	13
Ver las skills disponibles	13
Ver el detalle de un agente	13
Paso 9. Ejemplo completo de proyecto	13
Qué flujo de trabajo seguiría un alumno	14

Caso 1. Quiero que el agente implemente código	14
Caso 2. Quiero documentación	14
Caso 3. Quiero preparar un commit	14
Recomendaciones para alumnado	14
Errores frecuentes	14
Mezclar agente y skill	14
No registrar la ruta de skills	15
No incluir AGENTS.md en instructions	15
Crear reglas demasiado vagas	15
Describir un proyecto que no existe aún	15
Ejercicio Propuesto	15
Ejercicio: Configura tu primer proyecto con OpenCode	15
Objetivo	15
Pasos	16
Resultado esperado	16
Solución orientativa	16
Resumen	16
Siguiente paso	17
Referencias y recursos	17

Guía de Configuración de OpenCode

Guía Paso a Paso para Configurar OpenCode en un Proyecto

Objetivo

Aprender a preparar un proyecto para trabajar con OpenCode usando:

- un fichero `AGENTS.md` con reglas del proyecto,
- una carpeta `.opencode/` con configuración local,
- agentes especializados,
- skills reutilizables,
- e instrucciones adicionales para orientar al agente.

Esta guía está pensada para alumnado de programación que quiere usar agentes de IA para desarrollar proyectos con más orden, mejor contexto y menos improvisación.

Qué vas a aprender

Al terminar esta guía sabrás:

- qué carpetas y archivos necesita OpenCode,
- para qué sirve cada uno,
- cómo crear un agente,
- cómo crear una skill,
- cómo conectar todo con `opencode.json`,
- y cómo comprobar que OpenCode está leyendo correctamente tu configuración.

Requisitos

Antes de empezar, necesitas:

- tener OpenCode instalado,
- estar dentro de un proyecto real,
- tener nociones básicas de terminal,
- y saber qué tipo de ayuda quieres delegar en el agente.

OpenCode funciona mejor cuando el proyecto ya tiene una intención clara: por ejemplo, desarrollar una API, documentar un taller, revisar código o preparar commits.

Idea clave

OpenCode no trabaja bien solo con prompts sueltos.

Su verdadero potencial aparece cuando le das contexto estructurado. Ese contexto suele repartirse en cuatro piezas:

1. `AGENTS.md`: reglas generales del proyecto.
2. `.opencode/opencode.json`: configuración local de OpenCode.
3. `.opencode/agents/`: agentes especializados.
4. `.opencode/skills/`: habilidades reutilizables.

Estructura mínima recomendada

La estructura local básica de un proyecto con OpenCode puede ser esta:

```
mi-proyecto/
├── AGENTS.md
├── .opencode/
│   ├── opencode.json
│   ├── agents/
│   │   ├── developer.md
│   │   └── reviewer.md
│   ├── skills/
│   │   ├── kotlin-best-practices/
│   │   │   └── SKILL.md
│   │   └── professional-commits/
│   │       └── SKILL.md
│   └── instructions/
│       └── project.md
└── src/
```

Qué hace cada elemento

AGENTS.md

Es el documento principal de instrucciones del proyecto.

Aquí defines:

- el objetivo del repositorio,
- las reglas de trabajo,
- la tecnología usada,
- las convenciones de nombres,
- el estilo de testing,
- las restricciones importantes,
- y la relación entre agentes y skills.

Piensa en `AGENTS.md` como la guía docente del proyecto para la IA.

`.opencode/opencode.json`

Es el fichero de configuración local que indica a OpenCode:

- qué modelo usar por defecto,
- qué agente es el principal,
- qué permisos tiene,
- dónde están las skills,
- qué instrucciones debe cargar,
- y qué agentes existen en este proyecto.

`.opencode/agents/`

Aquí van los agentes especializados.

Cada agente tiene un rol concreto, por ejemplo:

- desarrollar código,
- redactar documentación,
- revisar cambios,
- o preparar commits.

`.opencode/skills/`

Aquí van las skills locales del proyecto.

Una skill no es un agente. Una skill es una guía reutilizable con criterios, buenas prácticas, reglas o plantillas para un tipo de trabajo.

Ejemplos:

- buenas prácticas de Kotlin,
- buenas prácticas con MongoDB,
- commits profesionales,
- testing con Kotest,
- documentación pedagógica.

`.opencode/instructions/`

Aquí puedes añadir instrucciones complementarias que no quieres meter enteras en `AGENTS.md`.

Por ejemplo:

- reglas del proyecto,
- decisiones de arquitectura,
- estilo de documentación,
- limitaciones técnicas.

Paso 1. Crear AGENTS.md

Empieza por un AGENTS.md sencillo y útil.

Ejemplo mínimo:

```
# AGENTS.md - Mi Proyecto
```

```
## Descripción
```

Este proyecto implementa una aplicación Kotlin para trabajar con MongoDB.

```
## Reglas
```

- Leer primero el código existente.
- Preferir cambios pequeños y correctos.
- No hardcodear credenciales.
- Usar nombres claros.

```
## Tecnología
```

- Kotlin
- Gradle Kotlin DSL
- MongoDB Atlas
- driver de MongoDB para Kotlin

```
## Skills del proyecto
```

- ``kotlin-best-practices``
- ``mongo-best-practices``

```
## Agentes del proyecto
```

- ``developer``
- ``git-committer``

Buenas prácticas para AGENTS.md

- Explica el proyecto antes de dar reglas.
- Distingue el estado actual del objetivo final si el proyecto aún está en construcción.
- No describas como existente una estructura que todavía no has creado.
- Usa reglas claras y comprobables.
- Indica dónde está la documentación fuente si existe.

Paso 2. Crear .opencode/opencode.json

Este archivo conecta toda la configuración.

Ejemplo mínimo funcional:

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "opencode/minimax-m2.5-free",
  "default_agent": "developer",
  "permission": {
    "bash": "ask",
    "read": "allow",
    "edit": "allow",
    "webfetch": "ask",
    "websearch": "ask",
    "skill": {
      "*": "allow"
    }
  },
  "watcher": {
    "ignore": [
      "**/node_modules/**",
      "**/build/**",
      ".git/**",
      "**/.gradle/**",
      "**/.idea/**"
    ]
  },
  "skills": {
    "paths": [
      ".opencode/skills"
    ]
  },
  "instructions": [
    "AGENTS.md",
    ".opencode/instructions/project.md"
  ],
  "agent": {
    "developer": {
      "description": "Desarrolla el proyecto principal.",
      "mode": "primary",
      "temperature": 0.2,
      "prompt": "{file:agents/developer.md}",
      "permission": {
        "bash": "ask",
        "read": "allow",
        "edit": "allow",
        "webfetch": "ask",
```



```
    "websearch": "ask",
    "skill": {
      "*": "allow"
    }
  },
  "git-committer": {
    "description": "Prepara commits profesionales.",
    "mode": "subagent",
    "temperature": 0.1,
    "prompt": "{file:agents/git-committer.md}",
    "permission": {
      "bash": "ask",
      "read": "allow",
      "edit": "deny",
      "webfetch": "deny",
      "websearch": "deny",
      "skill": {
        "professional-commits": "allow",
        "*": "ask"
      }
    }
  }
}
```

Paso 3. Entender los parámetros de `opencode.json`

`$schema`

Indica el esquema de configuración esperado.

```
"$schema": "https://opencode.ai/config.json"
```

`model`

Modelo por defecto que usará OpenCode.

```
"model": "opencode/minimax-m2.5-free"
```

`default_agent`

Agente principal del proyecto.

```
"default_agent": "developer"
```

permission

Permisos por defecto.

Ejemplo:

- bash: ask pide confirmación antes de ejecutar terminal.
- read: allow permite leer archivos.
- edit: allow permite editar.
- skill: { "*" : "allow" } permite cargar skills.

watcher.ignore

Sirve para indicar carpetas que OpenCode debe ignorar.

Es útil para no meter ruido de:

- node_modules
- build
- .git
- .gradle
- .idea

skills.paths

Le dice a OpenCode dónde buscar skills locales.

```
"skills": {  
  "paths": [  
    ".opencode/skills"  
  ]  
}
```

instructions

Lista de documentos que OpenCode debe cargar como instrucciones adicionales.

```
"instructions": [  
  "AGENTS.md",  
  ".opencode/instructions/project.md"  
]
```

agent

Define los agentes disponibles en el proyecto.

Cada entrada contiene normalmente:

- description
- mode

- temperature
- prompt
- permission

Paso 4. Crear un agente

Un agente es un archivo Markdown con instrucciones concretas.

Ejemplo: `.opencode/agents/developer.md`

```
---
name: Developer
description: Desarrolla código del proyecto siguiendo las reglas locales.
mode: subagent
model: OpenAI/GPT-5.4
temperature: 0.2
permission:
  bash: ask
  read: allow
  edit: allow
  webfetch: ask
  websearch: ask
  skill:
    kotlin-best-practices: allow
    mongo-best-practices: allow
    "*": ask
---
```

Developer

Rol

Eres un agente especializado en desarrollar el proyecto.

Flujo de trabajo

1. Lee primero el código y la documentación.
2. Aplica el cambio más pequeño correcto.
3. Usa las skills disponibles.
4. No inventes archivos ni clases que no existan sin justificarlo.

Qué partes debe tener un agente

Un agente útil suele tener:

1. Un bloque de metadatos.

2. Un rol claro.
3. Un contexto del proyecto.
4. Un flujo de trabajo.
5. Reglas importantes.
6. Skills recomendadas.

mode en agentes

Los valores más importantes que estás viendo en este proyecto son:

- `primary`: agente principal del proyecto.
- `subagent`: agente auxiliar o especializado.

Paso 5. Crear una skill

Una skill se define normalmente en una carpeta propia con un archivo `SKILL.md`.

Ejemplo: `.opencode/skills/kotlin-best-practices/SKILL.md`

```
---
name: kotlin-best-practices
description: "Buenas prácticas para escribir código Kotlin claro y mantenible."
---
```

```
# Kotlin Best Practices
```

```
## Reglas
```

- Preferir ``val`` sobre ``var``.
- Usar nombres expresivos.
- Mantener funciones pequeñas.
- Documentar APIs públicas cuando tenga sentido.
- Aplicar clean code y SOLID sin sobreingeniería.

Qué debe tener una skill

Una buena skill suele incluir:

- nombre,
- descripción,
- objetivo,
- reglas,
- ejemplos,
- anti-patronos,
- y criterios de uso.

Diferencia entre agente y skill

Usa esta idea para no confundirlos:

- el agente decide y ejecuta una estrategia de trabajo,
- la skill aporta criterio experto reutilizable.

Ejemplo:

- taller-developer es un agente,
- mongo-best-practices es una skill.

Paso 6. Añadir instrucciones complementarias

Ejemplo: `.opencode/instructions/project.md`

```
# Project Instructions
```

```
## Working Style
```

- Read the existing code before making changes.
- Prefer the smallest correct change.
- Keep examples easy to understand.

```
## Safety
```

- Do not hardcode secrets.
- Ask before destructive operations.

Este archivo sirve para reforzar reglas sin convertir `AGENTS.md` en un documento demasiado largo.

Paso 7. Arrancar OpenCode en el proyecto

Según la ayuda de OpenCode, puedes arrancarlo de varias formas.

Abrir la interfaz en el proyecto actual

```
opencode .
```

También puedes situarte dentro del proyecto y ejecutar:

```
opencode
```

Ejecutar una petición directa desde terminal

```
opencode run "revisa la estructura del proyecto"
```

Ejecutar con un agente concreto

```
opencode run --agent git-committer "prepara un commit para los cambios actuales"
```

Paso 8. Comprobar que OpenCode ha cargado la configuración

Estos comandos son especialmente útiles para alumnado porque permiten depurar la configuración.

Ver los agentes disponibles

```
opencode agent list
```

Ver la configuración resuelta

```
opencode debug config
```

Ver las skills disponibles

```
opencode debug skill
```

Ver el detalle de un agente

```
opencode debug agent git-committer
```

Paso 9. Ejemplo completo de proyecto

Supón que quieres un proyecto de Kotlin con MongoDB y tres apoyos de IA:

- un agente que programe,
- un agente que documente,
- y un agente que prepare commits.

La estructura podría ser esta:

```
mi-taller-mongo/  
├── AGENTS.md  
├── .opencode/  
│   ├── opencode.json  
│   ├── agents/  
│   │   ├── taller-developer.md  
│   │   ├── taller-documenter.md  
│   │   └── git-committer.md  
│   ├── skills/  
│   │   ├── kotlin-best-practices/  
│   │   │   └── SKILL.md  
│   │   ├── mongo-best-practices/  
│   │   │   └── SKILL.md  
│   │   └── professional-commits/  
│   │       └── SKILL.md  
│   └── instructions/  
│       └── project.md  
└── doc/
```

└─ src/

Qué flujo de trabajo seguiría un alumno

Caso 1. Quiero que el agente implemente código

1. El alumno abre el proyecto con opencode.
2. El agente principal lee AGENTS.md.
3. OpenCode carga opencode.json.
4. El agente taller-developer usa las skills de Kotlin y MongoDB.
5. El agente propone o implementa cambios con contexto del proyecto.

Caso 2. Quiero documentación

1. El alumno usa el agente de documentación.
2. Ese agente trabaja con reglas pedagógicas y estructura Markdown.
3. La documentación queda alineada con el proyecto, no improvisada.

Caso 3. Quiero preparar un commit

1. El alumno pide usar git-committer.
2. El agente revisa git status y git diff.
3. Propone un mensaje siguiendo Conventional Commits.
4. Espera confirmación antes de hacer el commit.

Recomendaciones para alumnado

- No empieces creando diez agentes. Empieza con uno o dos.
- No escribas prompts genéricos si puedes dejar reglas en archivos.
- Mantén cada agente con una responsabilidad clara.
- Convierte en skill aquello que quieras reutilizar en varios agentes.
- Usa AGENTS.md para explicar el proyecto de verdad, no solo para poner normas sueltas.
- Revisa con frecuencia opencode debug config si algo no se comporta como esperas.

Errores frecuentes

Mezclar agente y skill

Error:

- crear una skill que intenta comportarse como un agente completo.

Corrección:

- la skill aporta criterio,
- el agente dirige el trabajo.

No registrar la ruta de skills

Error:

- crear `.opencode/skills/` pero olvidar `skills.paths` en `opencode.json`.

Consecuencia:

- OpenCode no encuentra las skills locales.

No incluir AGENTS.md en instructions

Error:

- escribir buenas reglas, pero no cargarlas.

Corrección:

```
"instructions": [  
  "AGENTS.md",  
  ".opencode/instructions/project.md"  
]
```

Crear reglas demasiado vagas

Error:

- “programa bien”, “hazlo bonito”, “usa buenas prácticas”.

Corrección:

- concreta naming, testing, estructura y restricciones.

Describir un proyecto que no existe aún

Error:

- documentar como real una estructura que todavía no está implementada.

Corrección:

- separa “estado actual” y “estructura objetivo”.

Ejercicio Propuesto

Ejercicio: Configura tu primer proyecto con OpenCode

Objetivo

Preparar un proyecto con:

- AGENTS.md,

- `.opencode/opencode.json`,
- un agente `developer`,
- y una `skill project-best-practices`.

Pasos

1. Crea la estructura de carpetas.
2. Escribe un `AGENTS.md` simple.
3. Configura `opencode.json` con un agente por defecto.
4. Crea un agente `developer.md`.
5. Crea una `skill project-best-practices/SKILL.md`.
6. Ejecuta `opencode agent list`.
7. Ejecuta `opencode debug skill`.

Resultado esperado

OpenCode debe detectar:

- el agente principal,
- las skills locales,
- y las instrucciones del proyecto.

Solución orientativa

```
mi-proyecto/  
├── AGENTS.md  
└── .opencode/  
    ├── opencode.json  
    ├── agents/  
    │   └── developer.md  
    ├── skills/  
    │   └── project-best-practices/  
    │       └── SKILL.md  
    └── instructions/  
        └── project.md
```

Resumen

Para configurar un proyecto con OpenCode necesitas pensar en capas:

1. `AGENTS.md` para las reglas globales.
2. `opencode.json` para la configuración local.
3. agentes para roles especializados.
4. `skills` para criterio reutilizable.
5. instrucciones extra para afinar el comportamiento.

Si montas bien estas piezas, el agente deja de ser una caja negra y pasa a ser una herramienta de trabajo mucho más controlable.

Siguiente paso

Una vez dominada esta configuración básica, el siguiente paso natural es crear:

1. un agente desarrollador,
2. un agente documentador,
3. una skill de testing,
4. y una skill de commits profesionales.

Eso ya te permite usar OpenCode no solo para pedir ayuda, sino para organizar de forma seria el trabajo de programación.

Referencias y recursos

- [OpenCode Documentation](#)
- [Agent skills](#)